# Advanced CPU Architecture & Hardware Accelerators

## Final Project Assignment Definition

## RISCV-based MCU

**Hanan Ribo**

**01.07.26**

# Table of contents

# 1. Aim of the project

- Design, synthesis, and analysis of a simple (single-cycle architecture) RV32IM CPU core with Memory Mapped I/O, interrupt capability, and ***Serial communication peripheral (entitles to a 20% bonus)***
- ***Pipelined RV32IM core*** instead of a single-cycle core ***(entitles to a 10% bonus)***
- Understanding of CPU vs. MCU concepts, and FPGA embedded memory structures

# 2. Definition and prior knowledge

The aim of this project is to design an RV32IM-based MCU. The CPU will use a single-cycle RV32IM architecture. The digital design will be located on an FPGA Board. The RV32IM architecture is a Harvard architecture that increases throughput and simplifies logic.

# 3. Assignment definition

The architecture must include an RV32IM ISA-compatible CPU with data memory DTCM and program memory ITCM to host the application's data and code segments. The block diagram of an architecture is given in Figure 1. The top level and the RV32IM core must be structural. The design must be compiled and loaded onto an FPGA board for testing.

**Note:** Use push-button **KEY0 as a System RESET** (brings the PC to the first program instruction)

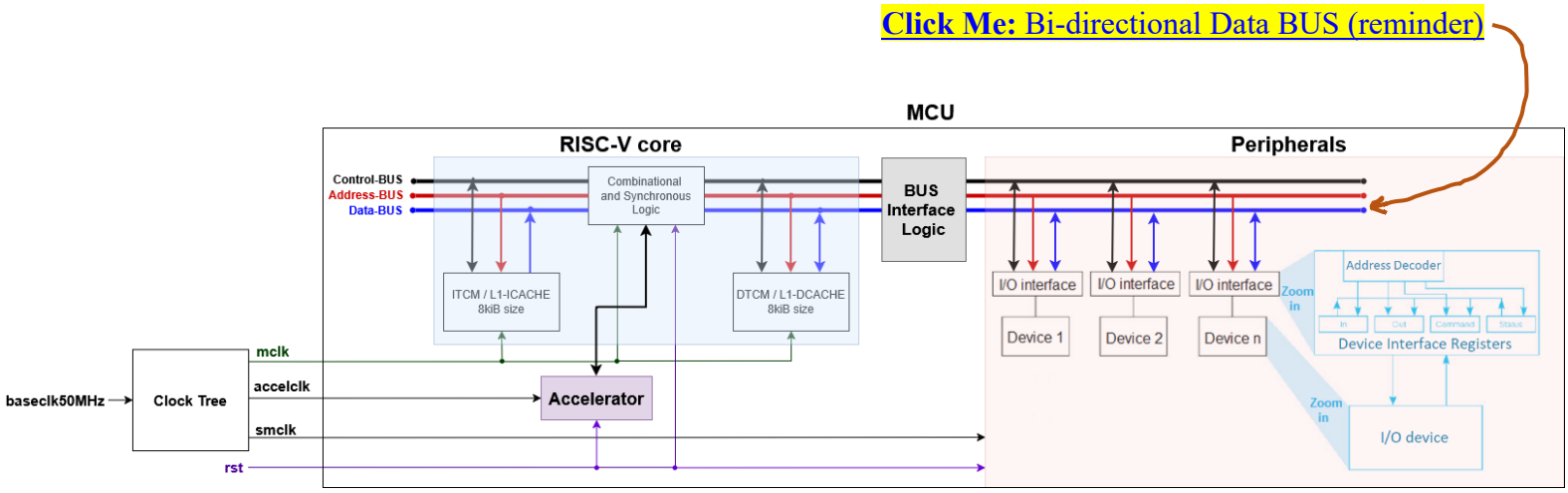



**Figure 1 :** MCU System General µArchitecture

The GPIO (General Purpose I/O) is a simple decoder with buffer registers mapped to data address (Higher than data memory) as given in the assembly code examples that enable the CPU to output data to GPIO devices such as LEDs-array, 7-Segment displays, and Switch-arrays. The Data Address Space addresses the DTCM (physical data memory) and I/O mapped devices at the lowest 14-bit address $\mathbf{0} \ldots \mathbf{0}A_{13} \ldots A_0$.

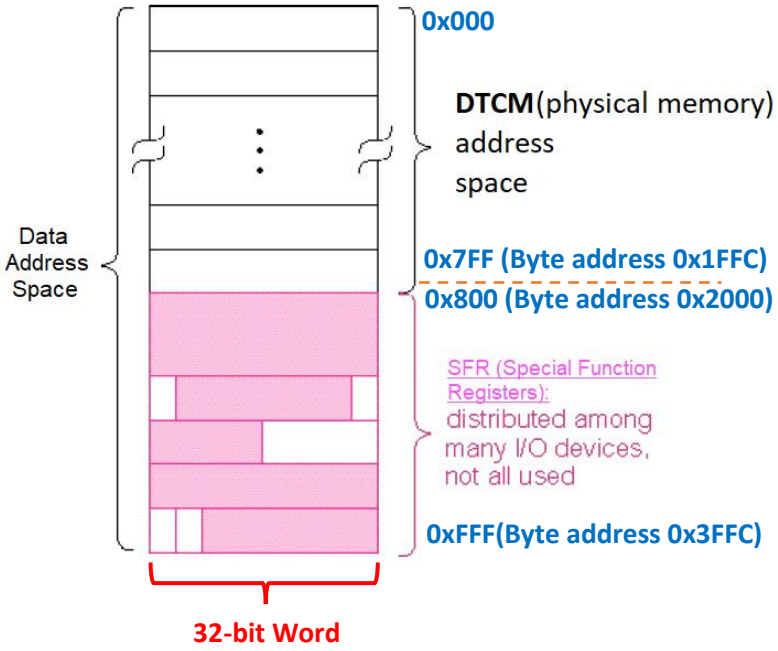



**Figure 2:** Data Address Space contains Data Memory and I/O Memory Mapped I/O

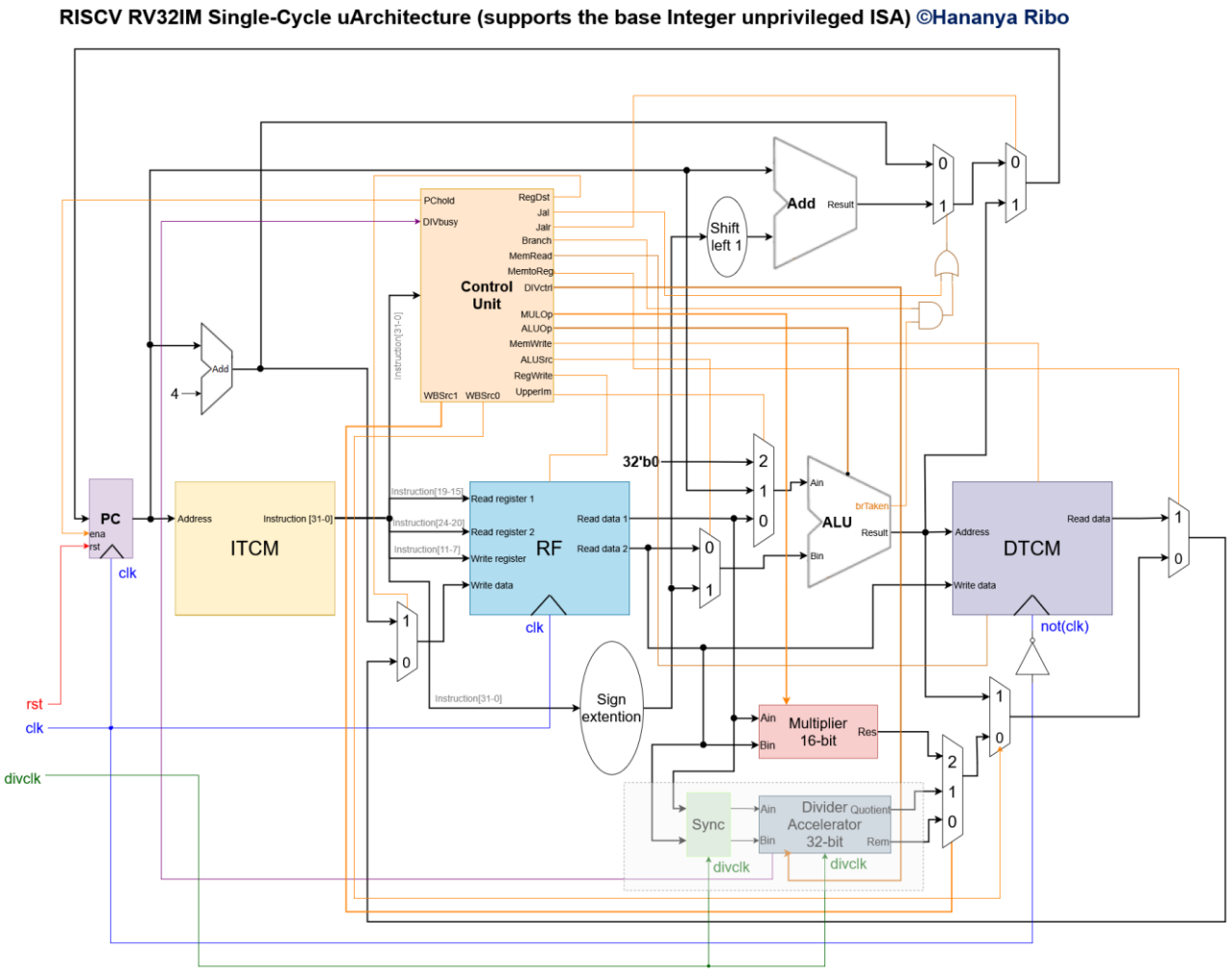



**Figure 3:** RV32IM single-cycle core connected to a divider accelerator

# 4. I/O devices:

- ***Ten*** switches (SW9-SW0) used as the ***input interface***.
- ***Four*** debounced pushbuttons (KEY3-KEY0) used as the ***input interface***.
- Board ***10*** red LEDs (LEDR9-LEDR0) used as ***Output interface***.
- ***Six*** 7-segment displays (HEX5-HEX0) used as ***Output interface***.
- Expansion Header GPIO of 2x20 pins

  Click Me: Figure 4a - I/O interface of the DE10-Standard FPGA board

  Click Me: Figure 4b - I/O interface of the DE2-115 FPGA board

# 5. Required Support of CPU's GPIO Peripherals:

### *Eight GPIO peripherals:*

| Memory-mapped GPIO without interrupt capability | | | | |
|---|---|---|---|---|
| **I/O device name** | **I/O mapping address** | **Note** | **Address Resolution** | **Direction** |
| PORT_LEDR | 0x2000 | LEDR7- LEDR0 | Byte address | GPO |
| PORT_HEX0 | 0x2004 | | Byte address | GPO |
| PORT_HEX1 | 0x2005 | | Byte address | GPO |
| PORT_HEX2 | 0x2008 | | Byte address | GPO |
| PORT_HEX3 | 0x2009 | | Byte address | GPO |
| PORT_HEX4 | 0x200C | | Byte address | GPO |
| PORT_HEX5 | 0x200D | | Byte address | GPO |
| PORT_SW | 0x2010 | SW7- SW0 | Byte address | GPI |

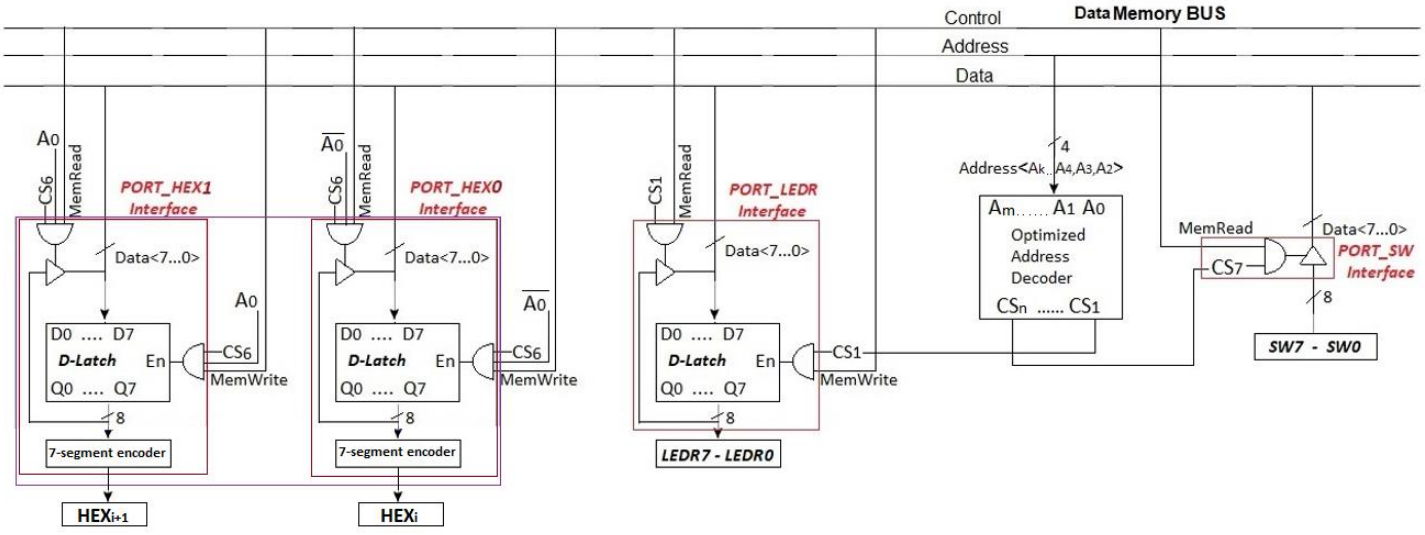



**Figure 5:** Basic GPIO peripheral connection using Memory Mapped I/O approach

# 6. Required Support of CPU's Peripherals with interrupt capability:

| Memory-mapped Peripherals with interrupt capability | | | |
|---|---|---|---|
| **I/O device name** | **I/O mapping address** | **Address Resolution** | **Device** |
| PORT_PB | 0x2014 | Byte address | KEY [3-1] |
| UTCL | 0x2018 | Byte address | USART |
| RXBF | 0x2019 | Byte address | USART |
| TXBF | 0x201A | Byte address | USART |
| BTCTL1 | 0x201C | Byte address | Basic Timer |
| BTCTL2 | 0x201D | Byte address | Basic Timer |
| BTCMPR0 | 0x2020 | Word address | Basic Timer |
| BTCMPR1 | 0x2024 | Word address | Basic Timer |
| BTCAPR | 0x2028 | Word address | Basic Timer |
| IE | 0x202C | Byte address | Interrupt Controller |
| IFG | 0x202D | Byte address | Interrupt Controller |
| TYPE | 0x202E | Byte address | Interrupt Controller |

**i.** KEY [3-1]:

support an array of ***three*** pushbuttons as an input device

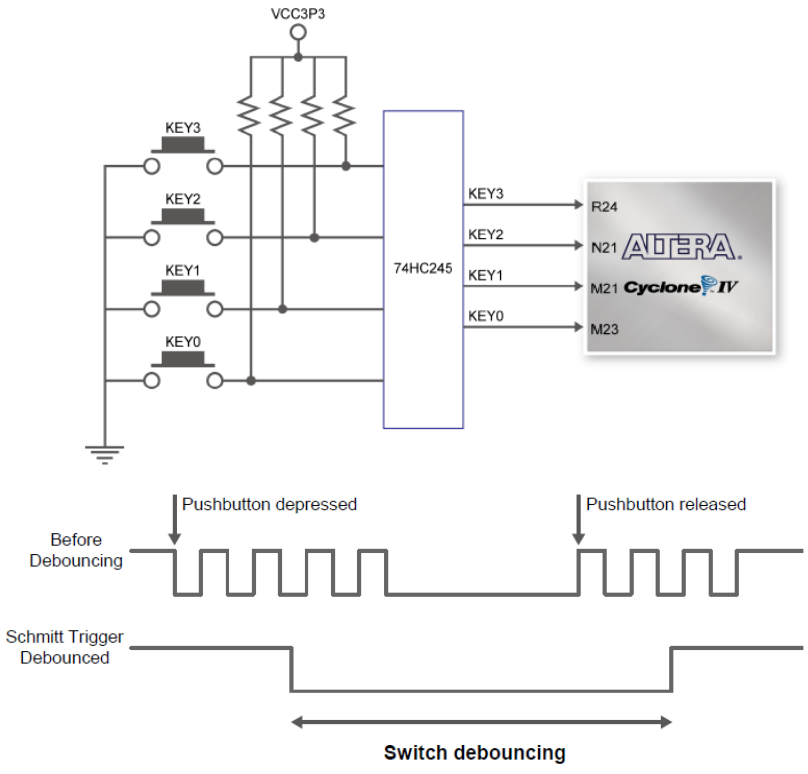
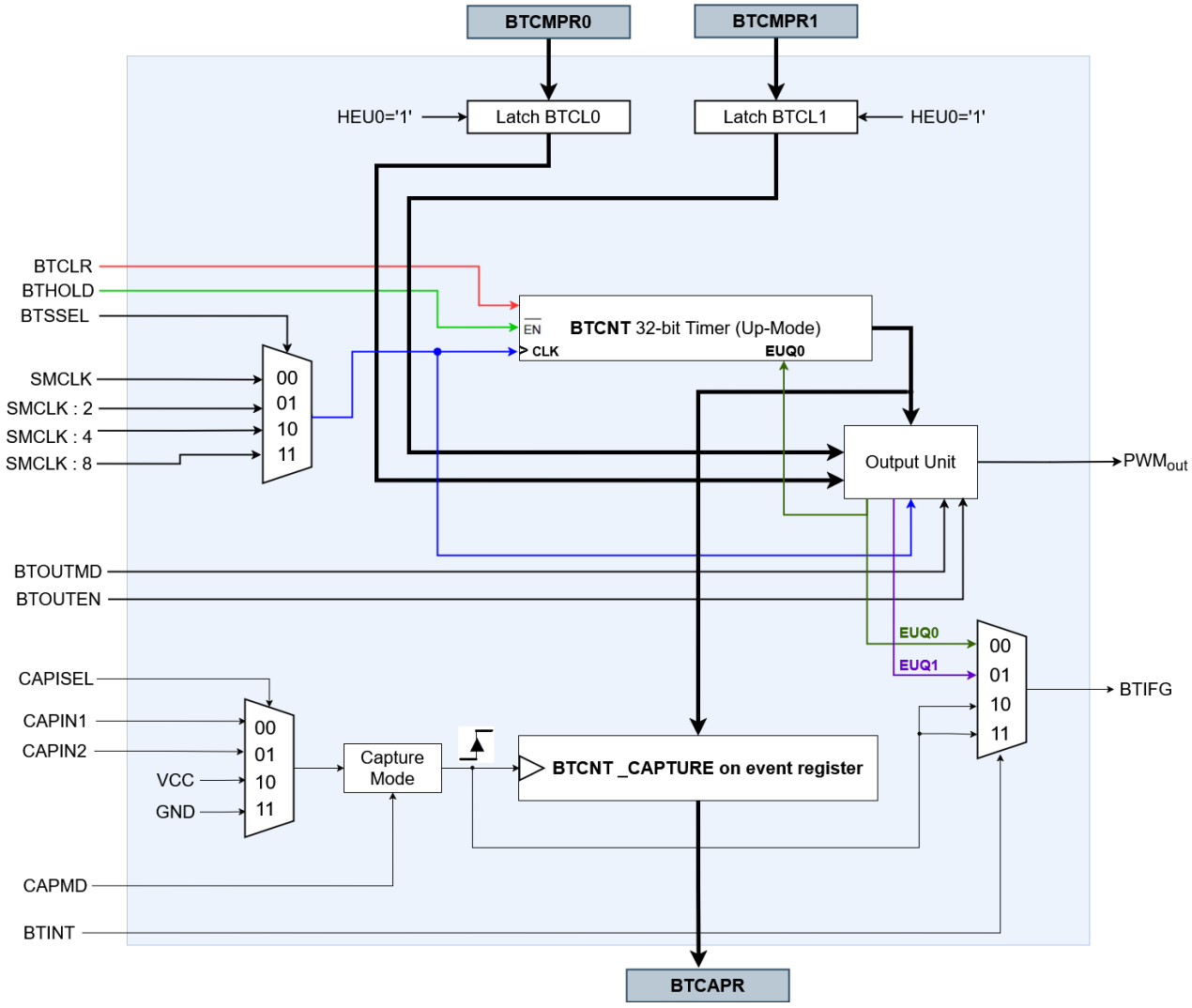



**Figure 6:** Pushbuttons hardware connection and debouncing

## ii. Basic Timer with output comparing capabilities:




**BTCTL1, Basic Timer Control Register1**

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| BTOUTMD | BTOUTEN | BTHOLD | BTSSEL | | BTCLR | BTINT | |
| rw | rw | rw | rw | rw | rw | rw | rw |

**BTCTL2, Basic Timer Control Register2**

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | CAPMD | | CAPISEL | |
| r | r | r | r | rw | rw | rw | rw |

**BTCAPR, Basic Timer Capture Register**

| 31 | 30 | . . . . . | 1 | 0 |
|---|---|---|---|---|
| BTCAPR | | | | |
| rw | rw | | rw | rw |

**BTCMPRx , Basic Timer Compare Register x = {0,1}**

| 31 | 30 | . . . . . | 1 | 0 |
|---|---|---|---|---|
| BTCMPRx | | | | |
| rw | rw | rw | rw | rw | rw | rw | rw |

- Data to be compared is written to each **BTCCRx** and automatically transferred to **BTCLx** (which holds the data for comparison with the timer register **BTCNT**). The register value is zero on RESET.

- The control register **BTCTL1** contains the following control bits:
  **BTINT**: select the Interrupt Source from three options
  **BTCLR**: reset the content of the BTCNT register
  **BTSSELL**: select the source clock of the Basic Timer
  **BTHOLD**: hold the content of the BTCNT register
  **BTOUTEN**: hold the PWMout signal value
  **BTOUTMD**: select the PWM output mode {0: Output Mode0; 1: Output Mode1}

- The control register **BTCTL2** contains the following control bits:
  **CAPISEL**: select the input capture signal source {0: CAPIN1 external input pin, 1: CAPIN2 external input pin, 2: VCC('1'), 3: GND('0')}
  **CAPMD**: select the input capture signal trigger {0,3: capture disabled, 1: Rising edge, 2: Falling edge}

- **Compare Mode:**
  The timer core BTCNT is compared to value of registers **BTCCRO0** or **BTCCR1** for producing periodic interrupt intervals.

- **Output Compare Mode:**
  The timer core BTCNT is compared to value of registers **BTCMPR0** and **BTCMPR1** for producing PWM signal.

- **Input Capture Mode:**
  The timer core BTCNT value is captured on event into register **BTCAPR**

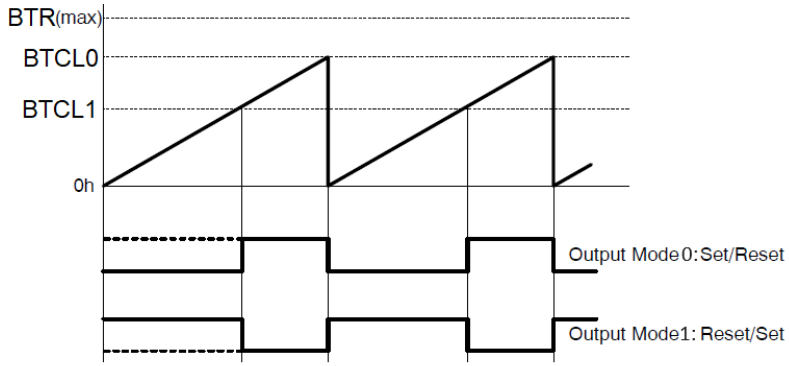



**Figure 8:** Output compare mode of PWM signals producing

### iii. Diagram of the Unsigned Binary Multicycle Division Accelerator:

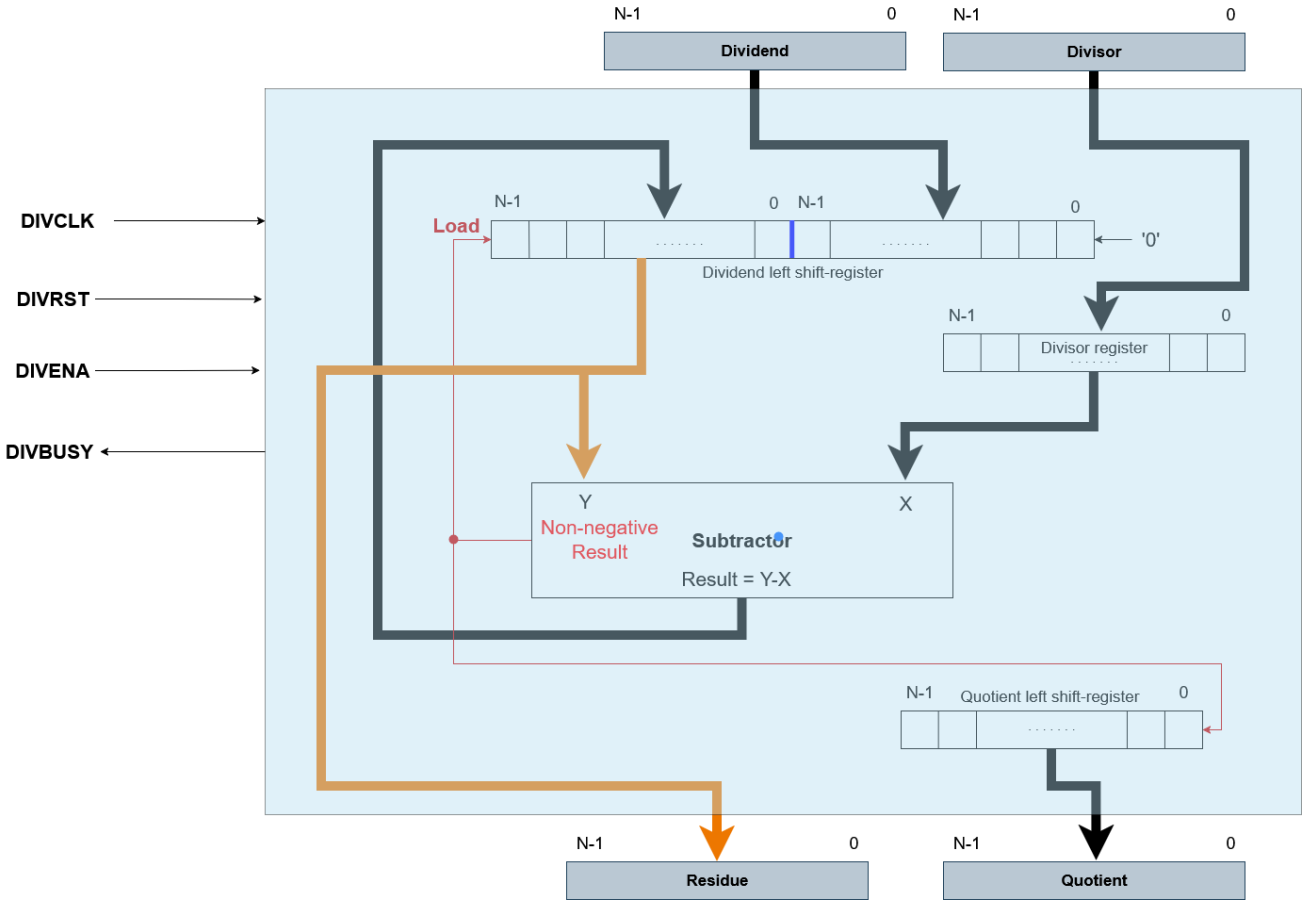
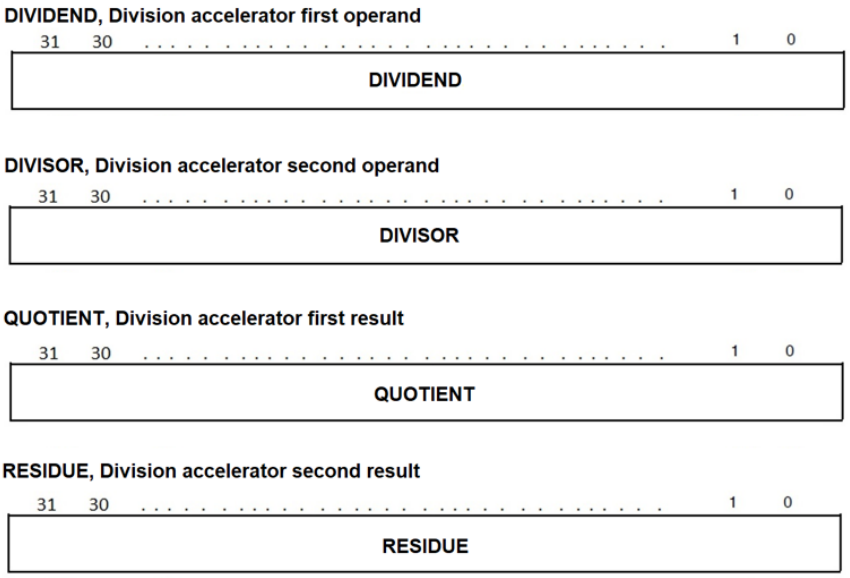



**Figure 9:** Multicycle-based architecture of Unsigned Divisor accelerator




The divider results are ready after N DIVCLK cycles after loading a value to the second operand DIVISOR, i.e., 32 DIVCLK cycles (**fast clock**) in our case of N=32. The Synchronizing approach is given in the Figure 10.

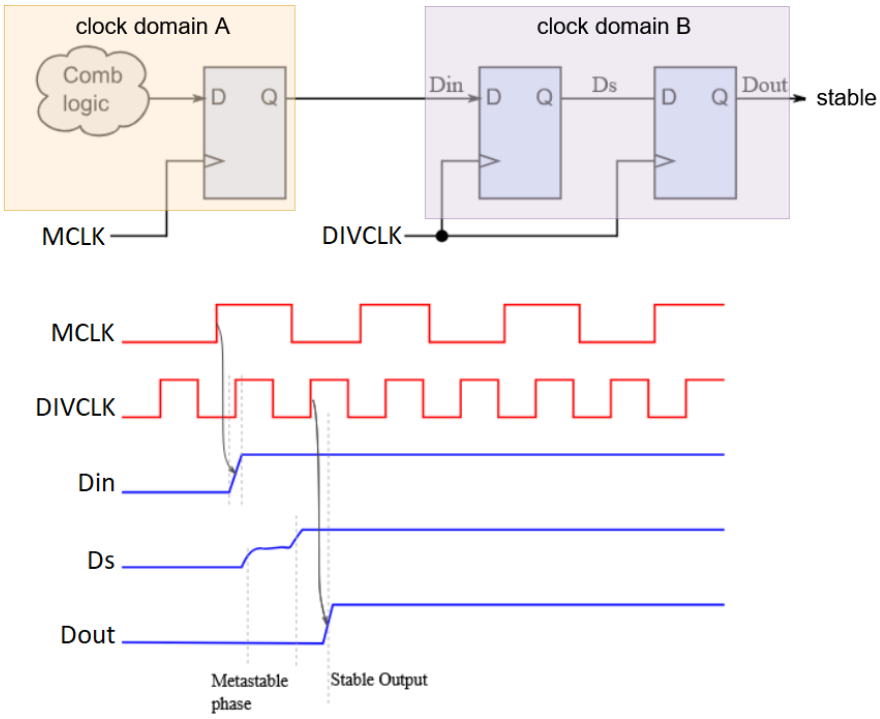



**Figure 10a:** CDC (clock domain crossing) synchronizer

In case a signal crosses clock boundaries from a slow clock domain (domain A) to a fast clock domain (domain B). It's fundamental to have a flip-flop to synchronize every signal that is driven by combinational logic (combo) in domain A before sending it to domain B through the synchronizer. In domain B, we must register the input to avoid metastability caused by violating the fast clock-domain B regime. In this case, the synchronizer architecture diagram is shown in Figure 10b.

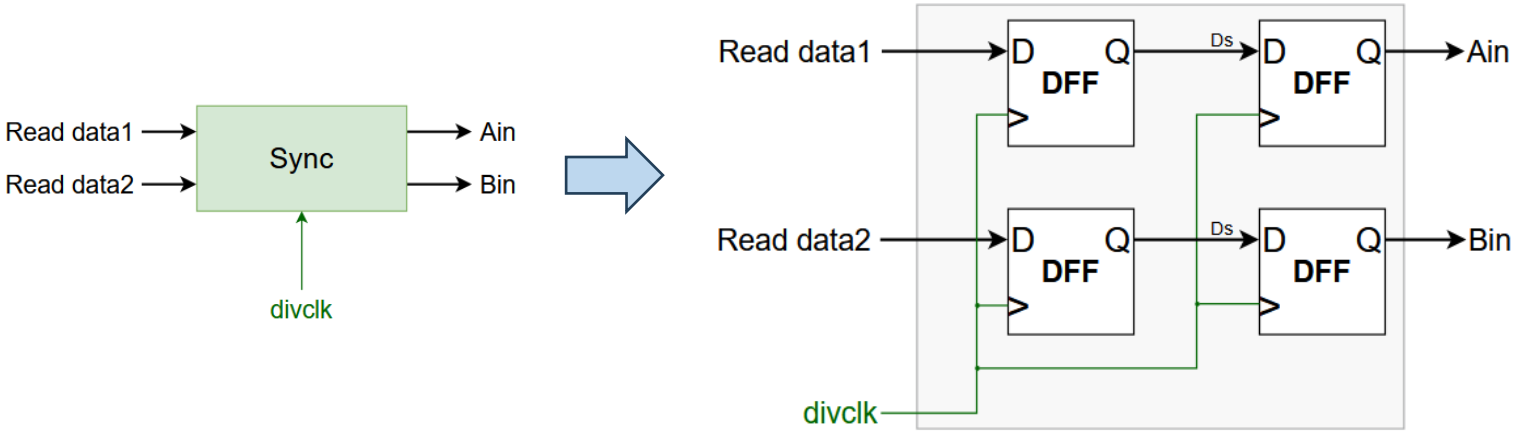



**Figure 10b:** The synchronizer architecture

### iv. USART Peripheral Interface, UART Mode (Bonus 20%):

The required communication peripheral is the universal **USART** (synchronous/asynchronous receive/transmit) peripheral interface in **UART** Mode only (degenerated **USART**).

You are given a VHDL design code that needs to be adapted to the following UART mode features.

*UART mode features include:*

- 1-start bit, 1-stop-bit, 8-bit data with non-parity
- Independent transmit and receive shift registers
- Separate transmit and receive buffer registers
- LSB-first data transmit and receive
- Programmable baud rate support
- Status flags for error detection
- Independent interrupt capability to receive and transmit

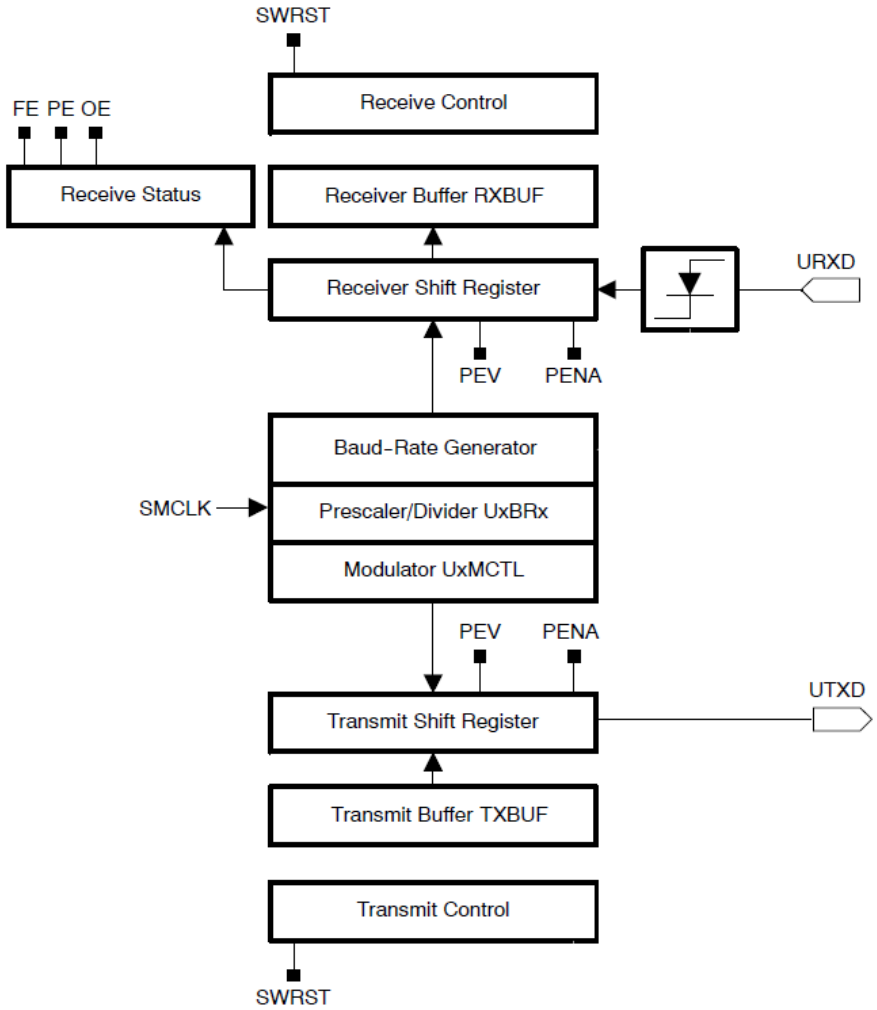



**Figure 11:** UART module functional diagram

### UCTL, USART Control Register

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| BUSY | OE | PE | FE | BAUDRATE | PEV | PENA | SWRST |
| r | r | r | r | rw | rw | rw | rw |

**SWRST** Bit 0 Software reset enable
0 Disabled. USART reset released for operation
1 Enabled. USART logic held in reset state

**PENA** Bit 1 Parity enable
0 Parity disabled
1 Parity enabled. Parity bit is generated (TXD) and expected (RXD).

**PEV** Bit 2 Parity select. PEV is not used when parity is disabled.
0 Odd parity
1 Even parity

**BAUDRATE** Bit 3 Baud Rate value
0 9600
1 115200

**FE** Bit 4 Framing error flag
0 No error
1 Character received with low stop bit

**PE** Bit 5 Parity error flag. When PENA = 0, PE is read as 0.
0 No error
1 Character received with parity error

**OE** Bit 6 Overrun error flag. This bit is set when a character is transferred into UxRXBUF before the previous character was read.
0 No error
1 Overrun error occurred

**BUSY** Bit 7 This bit indicates if a transmit or receive operation is in progress (busy).
0 UART module inactive
1 UART module transmitting or receiving

### RXBUF, USART Receive Buffer Register

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
| r | r | r | r | r | r | r | r |

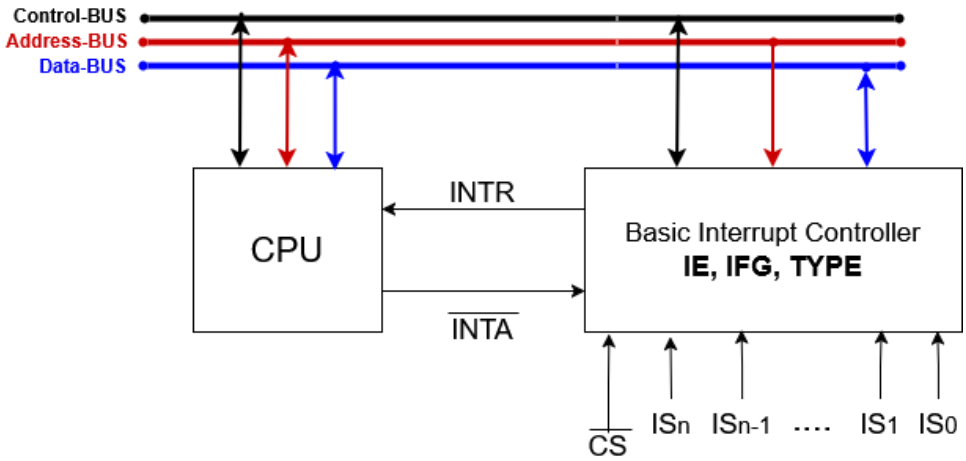
**RXBUFx** Bits 7-0 The receive-data buffer is user accessible and contains the last received character from the receive shift register. Reading RXBUF resets the receive-error bits, and RXIFG.

### TXBUF, USART Transmit Buffer Register

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| $2^7$ | $2^6$ | $2^5$ | $2^4$ | $2^3$ | $2^2$ | $2^1$ | $2^0$ |
| rw | rw | rw | rw | rw | rw | rw | rw |

**TXBUFx** Bits 7-0 The transmit data buffer is user accessible and holds the data waiting to be moved into the transmit shift register and transmitted on TXD. Writing to the transmit data buffer clears TXIFG.

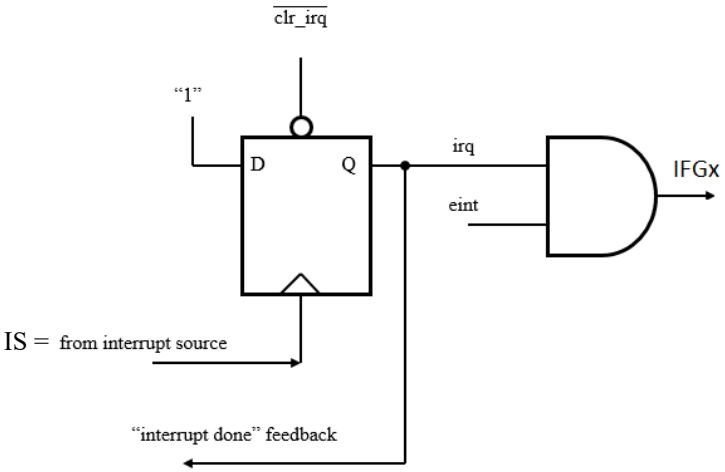
v. Interrupt controller:



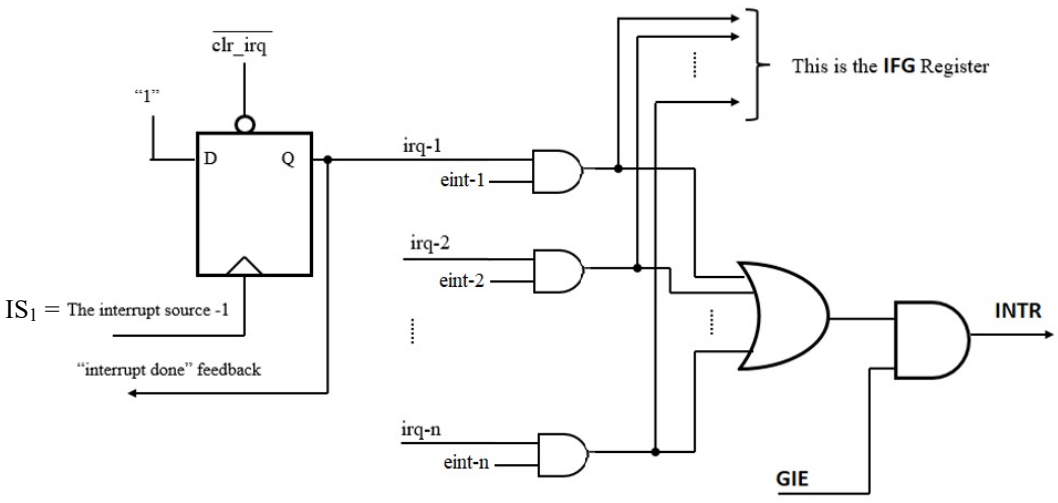

Notes:

*a.* The **BTIFG** flag is reset automatically when the interrupt is serviced.
*b.* **RXIFG** is automatically reset if the pending interrupt is served or when **RXBUF** is read.
*c.* **TXIFG** is automatically reset if the interrupt request is serviced or if a character is written to **TXBUF**
*d.* The **KEYiIFG** is reset manually with software (**BTIFG, RXIFG, TXIFG** as well).
*e.* As part of CPU servicing an interrupt, **GIE** is clear (***in HW***), which means **DINT** of other interrupts.
*f.* Symmetrically, as part of CPU returning from interrupt, **GIE** is set (***in HW***), which means **EINT** of interrupts (back to the original state).

## Handling an interrupt from a single source:




## Handling interrupts from several sources:



**IE, Interrupt Enable Register**

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | KEY3IE | KEY2IE | KEY1IE | BTIE | TXIE | RXIE |
| r-0 | rw | rw | rw | rw | rw | rw | rw |

IEx Bit x 0 Interrupt not enabled
1 Interrupt enabled

**IFG, Interrupt Flag Register**

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | KEY3IFG | KEY2IFG | KEY1IFG | BTIFG | TXIFG | RXIFG |
| r-0 | rw | rw | rw | rw | rw | rw | rw |

IFGx Bit x 0 No interrupt pending
1 Interrupt pending

**TYPE, Interrupt Type Register**

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| 0 | 0 | TYPEx | | | | | |
| r-0 | r-0 | r | r | r | r | r-0 | r-0 |

| Interrupts Vector Table | | | | |
|---|---|---|---|---|
| **Interrupt Source** | **Interrupt Flag** | **TYPE content** | **System Interrupt** | **Priority** |
| RESET | - | 00h | NMI | 0, Highest |
| UART status error | RXIFG | 04h | maskable | 1 |
| UART RX | RXIFG | 08h | maskable | 2 |
| UART TX | TXIFG | 0Ch | maskable | 3 |
| Basic Timer | BTIFG | 10h | maskable | 4 |
| Pushbutton 1 | KEY1 | 14h | maskable | 5 |
| Pushbutton 2 | KEY2 | 18h | maskable | 6 |
| Pushbutton 3 | KEY3 | 1Ch | maskable | 7, Lowest |

**Note:** NMI = Non Maskable Interrupt

## Protocol of Interrupt Service Process in a single-cycle CPU:

This multicycle-based control module is in the CPU's control unit. At the next clock cycle after **INTR** is set to '1' (by the interrupt controller), the **INTA** signal is set to '0' (by the CPU). At this point (before starting the interrupt service process), the next PC address is called the ***interrupt return address***.

1. CPU services an interrupt request (latency of two cycles):

   This multicycle process is triggered by the falling edge of the **INTA** signal (the ensuing cycle after **INTR** is set to '1')

   **Cycle 1:**

   - Clear GIE (bit gp[0] = 0)
   - Set INTA (INTA='1')
   - Writing the content of register TYPE on Data BUS and capturing it in a dedicated register.

     **Note:**
     cannot be written on the Address BUS because the CPU is the only BUS master (executes this protocol).

   **Cycle 2:**

   - In case of TYPE pending interrupt of a synchronous interrupt source, clear its flag (like the BTIFG)
   - Emulation execution of ***load*** (of TYPE content) and ***jalr*** (to Mem [TYPE] content) where R[tp]=interrupt return address.

2. CPU returning from service of an interrupt request (latency of one cycle):

   As a part of the execution of ***jalr* zero, 0(tp)** (*return from interrupt = reti)*, set GIE (bit gp[0] = 1).

## 6. PPA characterization of RV32IM-based MCU design:

| Area | | | | | | |
|---|---|---|---|---|---|---|
| | Logic elements | Registers | I/O pins | Embedded memory bits | Embedded 9-bit Multipliers | PLLs |
| MCU with GPIO | | | | | | |
| MCU with GPIO and Interrupt Capability | | | | | | |
| Pipelined MCU with GPIO and Interrupt Capability | | | | | | |

**Note:** Attaching the print screen of the Quartus Arae report is mandatory

| Performance | | | |
|---|---|---|---|
| | $f_{max}$ | Critical path (what is the slowest submodule and why does it cause the critical path) | $f_{MCLK}$ $(= f_{sysclk})$ |
| MCU with GPIO | | | |
| MCU with GPIO and Interrupt Capability | | | |
| Pipelined MCU with GPIO and Interrupt Capability | | | |

**Note:** Attaching the print screen of the Quartus *fmax* report is mandatory

| Power | | | | |
|---|---|---|---|---|
| | **Total** power consumption | **Static** power consumption | **Dynamic** power consumption | **I/O** power consumption |
| MCU with GPIO | | | | |
| MCU with GPIO and Interrupt Capability | | | | |
| Pipelined MCU with GPIO and Interrupt Capability | | | | |

**Note:** Attaching the print screen of the Quartus Power report is mandatory

## 7. Pin Planner

Only MCU IO devices need to be connected to FPGA location pins via the pin planner. Location pins used for the validation phase (Signal-Tap) need to be removed in the final step using a suitable parameter in the ***generate*** VHDL statement.

***Recall that validation using Signal-Tap is mandatory!***

## 8. RV32IM-based MCU design flow based on benchmark applications:

***a)*** Characterize the architecture design in detail

***b)*** Write HDL code for architecture design

***c)*** Verify your design using ModelSim based on benchmark applications.

i. As a golden model, at the end of the application run, compare the RARS output file DTCM.h with the ModelSim output file DTCM.mem.

ii. For full coverage, use basic and advanced benchmark applications.

iii. Check the IPC by comparing the two sides of the equation in clause 6.iii.b (IPC accuracy matters).

iv. In case of inequality in ii or iii, **return to phase (b).**

***d)*** Use Quartus with *.sdc file for PPA analysis (see three tables in clause 7).

***e)*** Validate the design using ISMCE and Signal-Tap validation tools.

i. As a golden model, at the end of the application run, compare the RARS output file DTCM.hex with the ISMCE output file DTCM.hex.

ii. For full coverage, use basic and advanced benchmark applications.

iii. Check the IPC by comparing the two sides of the equation in clause 6.iii.b (IPC accuracy matters).

iv. In case of inequality in ii or iii, **return to phase (b).**

## 8. UART bonus - benchmark applications:

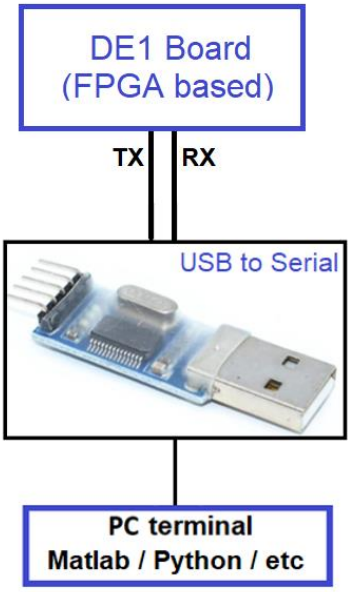
Using serial communication support application of PC side (like ***Hyper-Terminal***, ***Tera-Term***, ***puTTY,*** etc) and a suitable application for the MCU side that supports the next menu (transmitted from MCU to PC):

Menu

1. Count from 0x00 onto LEDG with delay ~0.5sec
2. Count down from 0xFF onto LEDR with delay ~0.5sec
3. Clear all LEDs
4. On each KEY1 pressed, send the message “I love my Negev”
5. Show Menu

## 9. UART bonus - MCU and PC communication using RS-232 interface:

The FTDI driver emulates a standard PC serial port such that the USB device may be communicated with as a standard RS-232 device. The driver allows direct access to a USB device via a DLL interface.




## 10. Requirements

The following must be presented in the **final.pdf** report file.

1. Top-level block review diagram of your design.
2. RTL Viewer results
3. Three tables in clause 7 with their shot screen report (from Quartus).
4. Short description of each HDL source file.
5. Documentation Style - Content with page numbers, images, and tables will be numbered. The caption of an image and tables below the image or table.
6. Elaborated analysis and waveforms:

   A basic waveform for benchmark applications test4-test1to explain the system timing.
7. Conclusions
8. Submission requirements:

   A ZIP file in the form of **id1_id2.zip** (where id1 and id2 are the identification numbers of the submitters, and id1 < id2) *must be uploaded to Moodle only by the student with id1* (any of these rule violations disqualifies the task submission).

The **ZIP** file will contain the next six subdirectories (***only the exact next sub folders***):

| Directory | Contains | Comments |
|---|---|---|
| DUT | Two subfolders, each containing design VHDL files:<br>• **RV32IMscMCU**<br>• Bonus only: **RV32IMpipelinedMCU** | Only the design VHDL files (excluding test bench).<br>Note: your project files must be well compiled (in ModelSim and Quartus separately) without errors as a basic condition before submission |
| TB | Two subfolders, each containing the design testbench file:<br>• **RV32IMscMCU**<br>• Bonus only: **RV32IMpipelinedMCU** | • In folder **RV32IMscMCU** insert the **tb_ RV32IMscMCU.vhd** file for RV32IM single-cycle based MCU design.<br>• In folder **RV32IMpipelinedMCU,** insert the **tb_ RV32IMpipelinedMCU.vhd** file for RV32IM pipelined based MCU design. |
| SIM | Two subfolders, each containing the ModelSim *.do file:<br>• **RV32IMscMCU**<br>• Bonus only: **RV32IMpipelinedMCU** | |
| DOC | Project documentation | • **Readme.txt** description of the content of each folder (and subfolder) for user convenience navigation.<br>• **Final_report.pdf** full report file described in clause 9 |
| Quartus | Two subfolders, each containing a Signal-Tap file, an SDC file, and a SOF file:<br>• **RV32IMscMCU**<br>• **RV32IMpipelinedMCU** | Do not place files that are not relevant for compilation or a result of compilation! |

**Table 1 : Directory Structure**

# 11. Grading policy

| Weight | Task | Description |
|---|---|---|
| 10% | Full Documentation | The "clear" way in which you presented the requirements, the analysis, and the conclusions on the work you've done |
| 90% | System Execution, Analysis, and Test | The correct analysis of the system (under the requirements) |

**Table 4: Grading**